FOL: A Functional Object Lisp

Frank Adrian
Independent Researcher
Portland, Oregon, USA
frank.adrian@gmail.com

***Abstract*—We present FOL (Functional Object Lisp), a new Lisp dialect that combines persistent functional data structures from Clojure with CLOS-style object-oriented programming and Dylan-inspired modules and naming conventions. FOL features a bootstrap implementation in Common Lisp, leveraging the FSet and Sycamore libraries for persistent collections. The language provides Clojure-compatible syntax and semantics for items like sequence operations and transducers while adding multiple destructuring pattern dispatch and maintaining full compatibility with CLOS's meta-object protocol. We demonstrate how FOL bridges concepts from multiple Lisp traditions and present a self-hosted meta-circular evaluator that showcases the language's metaprogramming capabilities.**

***Index Terms*—Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers**

I. Introduction

The Lisp family of languages has evolved along multiple paths, each emphasizing different aspects of the language's core philosophy. Common Lisp [**?**] provides a mature, standardized platform with powerful object-oriented features through CLOS [**?**] and metaprogramming through the MOP [**?**]. Clojure [**?**] introduced persistent data structures and functional programming idioms to the JVM ecosystem, emphasizing immutability and simplicity. Dylan [**?**] explored an object-oriented Lisp with a clear type system. Clojure provides objects, but this facility is constrained by what is easily provided by the JVM and the language itself. We believe CLOS and its MOP provides a much richer object system. FOL exists to show that immutable persistent objects and an object system like CLOS can co-exist and, in fact, can be synergistic when used together. In addition, FOL provides extended multiple destructuring patterns in functions, CLOS generic functions and methods, and macros, providing a richer programming experience for the user.

FOL (Functional Object Lisp) synthesizes ideas from these traditions to create a language that:

- Provides Clojure-style persistent data structures with structural sharing for all objects not requiring mutability (exceptions are streams, atoms, regular expression scanners, etc.)
- Maintains CLOS-style generic functions including multiple destructuring pattern dispatch
- Adopts Dylan's naming conventions (`<type>`) for clarity and its module structure which is simpler and more intuitive than Clojure's namespaces or Common Lisp's packages.
- Supports both functional and object-oriented programming paradigms

This paper describes FOL's design, implementation, and contributions to the Lisp ecosystem.

II. Language Design

A. Core Philosophy

FOL is built on the principle that persistent data structures and object-oriented programming are complementary, not antagonistic. While Clojure eschews traditional OOP in favor of protocols and multi-methods, FOL embraces CLOS's full power while maintaining Clojure's functional purity through persistence.

The language makes the following design commitments:

1) **Immutability by default**: All objects and collections are persistent for all objects not requiring mutability (e.g., streams, atoms, regular expression scanners, etc.)
2) **Structural sharing**: Updates to objects and collections create new versions sharing structure with old
3) **Object identity**: Objects have identity separate from value equality
4) **Generic functions**: Multiple dispatch over destructuring patterns for polymorphism
5) **Meta-object protocol**: Full introspection and extension capabilities

These commitments address fundamental tensions in language design. Immutability provides referential transparency and thread safety, essential for concurrent programming and reasoning about program behavior. However, naive immutable implementations copying entire data structures on each modification would be prohibitively expensive. Structural sharing resolves this tension by making immutability practical—updates sharing unchanged portions with previous versions achieve near-constant time performance for many operations.

The inclusion of object identity alongside value equality acknowledges that real-world programs model entities with identity that persists across state changes. A person aging from 30 to 31 years remains the same person despite changed attributes. FOL captures this through persistent objects that maintain identity while creating new versions on modification, bridging functional programming's emphasis on values with object-oriented programming's modeling of stateful entities.

Generic functions with destructuring pattern dispatch extend CLOS's multiple dispatch with pattern matching capabilities found in functional languages. This combination enables

polymorphism based on both type information and structural properties, providing more expressive method specialization than either approach alone. The meta-object protocol ensures these mechanisms remain open to programmer extension and customization.

B. Syntax and Readability

FOL adopts Clojure's reader syntax for consistency with functional programming conventions:

```
;; Vectors
[1 2 3]

;; Maps (dictionaries)
{:name "Alice" :age 30}

;; Sets
#{1 2 3 4}

;; Destructuring definition with predicate tests
(defn factorial
  ([(n (<= 1))] 1)
  ([n]
    (* n (factorial (dec n)))))

;; Destructuring with type constraints and :as
(defn summarize-person
  ([{:keys [(name <string>) (age <number>)]
     :as person}]
    (str name " is " age " years old")))

;; Rest parameters with &
(defn sum-and-product
  ([fst & rst]
    (make <dict>
      :sum (+ fst (apply + rst))
      :product (* fst (apply * rst)))))

;; Nested destructuring with :or for defaults
(defn process-config
  ([{:keys [server port]
     :or {port 8080}
     :as config}]
    (bind [{:keys [host timeout]
            :or {timeout 30}} server]
      {:host host
       :port port
       :timeout timeout})))
```

These examples demonstrate FOL's rich destructuring capabilities, combining pattern matching with predicate tests, default values, and nested destructuring. The use of angle brackets (<>) for type names, borrowed from Dylan, provides visual distinction between types and values.

C. Type System

Although a small number of system types must be mutable and derive from CLOS's standard-object, the majority of FOL's type hierarchy begins with `<persistent-object>` from which all primitive types, collection types, and user types derive.

Primitive types wrap their Common Lisp equivalents in a `val` slot, accessed through the generic function `fol-val`. This allows primitives to participate in the persistent object protocol while maintaining compatibility with Common Lisp's numeric tower.

III. Architecture and Implementation

A. Bootstrap Implementation

The implementation consists of approximately 6,000 lines of Common Lisp code:

TABLE I
Implementation File Summary

File	LOC	Purpose
persistent	250	Base persistent object protocol
classes	180	Primitive type wrappers
collection	890	Persistent collections
seqop	1420	Sequence operations
eval	1580	Evaluator and special forms
standard-names	1680	Standard environment

The code has been thoroughly tested with over 6,300 tests passing across the bootstrap implementation.

B. Persistent Object Protocol

FOL extends CLOS with a persistent object protocol. All user-defined classes inherit from `<persistent-object>` and use `pslot-value` for slot access (although, as in the code below, one can also use the function `assoc` to assign new values, access values via a keyword having the slot-name, or call the `get` function to retrieve the value of a slot):

```
(defclass <person> [<persistent-object>]
  [[name :type <string>]
   [age :type <number>]])

(def alice (make <person> :name "Alice" :age 30))

;; Updates return new instances via assoc
(def older-alice
  (assoc alice :age 31))

;; Original unchanged
(:age alice)     ; => 30
(older-alice :age)  ; => 31
(get alice :name)   ; => "Alice"
```

FOL's `defclass` macro extends CLOS's `defclass` to create persistent classes. It ensures all instances inherit from `<persistent-object>` and automatically implements the persistent slot protocol. Like CLOS, it supports slot options including `:type`, `:initarg`, `:initform`, and `:reader`/`:writer` specifications. Writer functions return a new object instance sharing structure with the original object with the value of the slot changed.

Persistence is achieved through careful management of slot values and structural sharing. When a slot is updated, only the affected slots are copied; others are shared between instances. This implementation strategy required solving several technical challenges. First, CLOS's native slot access mechanisms (`slot-value` and `setf`) assume mutable slots, necessitating a parallel protocol (`pslot-value` and `set-pslot-value`) that returns new instances rather than modifying existing ones.

Second, integrating persistent objects with CLOS's meta-object protocol required careful attention to initialization and

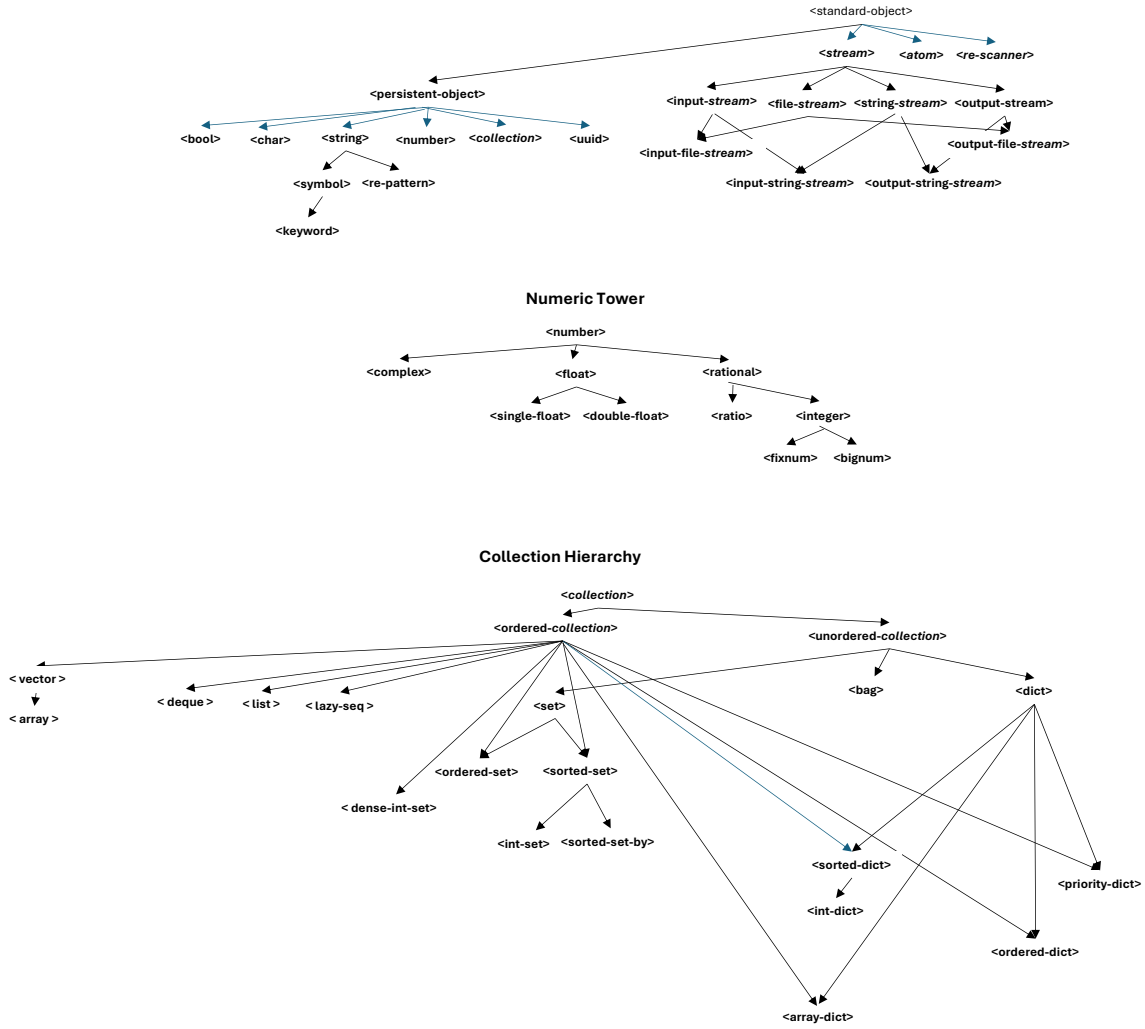

Fig. 1. FOL Type Hierarchy showing inheritance from `<persistent-object>`.

allocation. Instance creation must allocate slot storage that can be efficiently shared across versions. Our implementation achieves this by storing slot values in a persistent hash map, allowing O(log n) updates and structural sharing at the slot level. This design choice prioritizes flexibility and correctness over raw performance, accepting logarithmic overhead to maintain the abstraction's integrity. The use of the persistent hash map also allows a user to add additional key-value pairs to any object.

The persistent object protocol also interacts with Common Lisp's garbage collector in interesting ways. Because old versions of objects remain reachable as long as any reference exists, programs must be mindful of retention. However, this same property enables time-travel debugging and undo/redo functionality essentially for free—simply retain references to previous versions. This capability has proven valuable in interactive development environments where programmers frequently experiment with state modifications.

C. Collection Implementation

FOL's collections are implemented as thin wrappers around FSet [**?**] and Sycamore data structures:

```
(defclass <collection> [<persistent-object>]
  [[items :type fset:collection]])

(defclass <vector> [<collection>] [])
(defclass <dict> [<collection>] [])
(defclass <set> [<collection>] [])
```

This design provides several benefits:

- Structural sharing inherent to FSet
- O(log n) updates for most operations
- Compatibility with Common Lisp sequence functions

- Extensibility through CLOS

The decision to wrap FSet rather than implement persistent collections from scratch reflects a pragmatic engineering philosophy. FSet provides battle-tested implementations of 2-3 finger trees and other sophisticated data structures, leveraging decades of research in purely functional data structures. Building on this foundation allowed FOL development to focus on language design and integration challenges rather than low-level collection implementation details.

This layered architecture also provides flexibility for future optimization. The thin wrapper layer isolates FOL code from FSet internals, allowing transparent replacement of underlying implementations. Alternative backends could be swapped in for specific use cases—for instance, using Sycamore's weight-balanced trees when maintaining sorted order is essential, or custom implementations optimized for specific access patterns.

The wrapper layer also addresses impedance mismatches between Common Lisp's and FOL's semantics. FSet operates on Common Lisp values, while FOL wraps primitives in persistent objects. The collection wrappers handle conversions transparently, presenting a uniform interface where all elements are FOL values regardless of underlying representation. This uniformity simplifies generic function implementations and enables consistent behavior across collection types.

IV. Features

A. Clojure Features

FOL implements several Clojure features including:

- **Transducers**: FOL implements Clojure's transducer protocol [**?**], enabling composable algorithmic transformations.
- **Lazy sequences**: Laziness is essential in efficient processing of sequences of objects and processing infinite sequences. FOL provides facilities for creating and processing lazy sequences.
- **Atoms**: Atoms are one of the most widely used storage options in Clojure. We provide thread-safe atoms for storage and atomic updating of state.
- **Collection hierarchy**: FOL adopts Clojure's rich set of collections. Sets and dicts have specializations that allow them to be sorted and provide efficient means to process integer-keyed collections.
- **Sequence functions**: In addition to transducers, FOL provides Clojure's wide-ranging set of functions operating on sequences.

B. Generic Function Integration

FOL's generic functions fully integrate with persistent data. Methods dispatch on type predicates (e.g., `<vector>?`, `<dict>?`) which check both the specified type and all subtypes, providing flexible polymorphism:

```
(defgeneric process [x])

(defmethod process [(x <vector>)]
  (vec (map process x)))

(defmethod process [(x <dict>)]
  (update-vals x process))

(defmethod process [(x <number>)]
  (* x 2))

;; Works recursively on nested structures
(process [1 {:a 2 :b 3} [[4]]])
; => [2 {:a 4 :b 6} [[8]]]

;; Multiple dispatch with type specialization
(defgeneric compute-total [items discount])

;; Dispatch on collection type
(defmethod compute-total
  [(items <vector>) (<number>? discount)]
  (* (reduce + items) (- 1.0 discount)))

;; Dispatch with map destructuring in body
(defmethod compute-total
  [(items <dict>) (discount <dict>)]
  (bind [{:keys [(rate <number>)
                 (type <keyword>)]
          :or {type :percentage}} discount
         subtotal (reduce + (vals items))]
    (if (= type :percentage)
      (* subtotal (- 1.0 rate))
      (- subtotal rate))))
```

In addition, one can specify a set of destructuring patterns in `defgeneric` and a set of patterns with their associated code in functions, methods and macros:

```
(defgeneric process ([a]
                     [a & (b (< 10))]
                     [a & (b <number>) (c <string>)
                         ]
                     [a & b c]))

(defmethod process
  ([a] 50)
  ([a (b (< 10))] (* 2 b))
  ([a (b <number>) (c <string>)] c)
  ([a b c] :error))
```

This demonstrates how generic functions provide polymorphism through multiple dispatch while maintaining destructuring capabilities and functional purity.

C. Predicate Specializers

FOL extends pattern matching with general predicate specializers that allow functions to dispatch based on arbitrary predicate tests. The syntax `(var (fn arg0 arg1 ...))` applies the predicate function to the argument at runtime: `(apply fn var arg0 arg1 ...)`.

1) Basic Predicate Specialization: Predicates work with any function, providing flexible dispatch:

```
;; Equality predicates
(defn check-value
  ([(n (= 0))] :zero)
  ([(n (= 1))] :one)
  ([n] :other))

;; Comparison predicates
(defn classify-number
  ([(n (< 0))] :negative)
  ([(n (= 0))] :zero)
  ([(n (> 0))] :positive))
```

```
;; Range checking
(defn age-group
  ([(age (< 13))] :child)
  ([(age (< 20))] :teen)
  ([(age (< 65))] :adult)
  ([age] :senior))

;; Symbol matching with quoted values
(defn dispatch-command
  ([(cmd (= 'start))] (start-system))
  ([(cmd (= 'stop))] (stop-system))
  ([(cmd (= 'restart))] (restart-system))
  ([cmd] (unknown-command cmd)))
```

2) Multiple Parameter Predicates: Predicate specializers support multiple parameters with independent predicates:

```
(defn compare-signs
  ([(a (< 0)) (b (> 0))] :negative-positive)
  ([(a (> 0)) (b (< 0))] :positive-negative)
  ([(a (= 0)) (b (= 0))] :both-zero)
  ([a b] :other))

(compare-signs -5 10)  ; => :negative-positive
(compare-signs 10 -5)  ; => :positive-negative
```

3) Custom Predicates: Any function can serve as a predicate, enabling domain-specific dispatch:

```
;; Type predicates
(defn process-value
  ([(x (<number>?))] (* x 2))
  ([(x (<string>?))] (str x x))
  ([(x (<vector>?))] (vec (map process-value x)))
  ([x] x))

;; Using standard even?/odd? predicates
(defn classify-parity
  ([(n (even?))] :even)
  ([(n (odd?))] :odd))
```

4) Predicate Specificity: Predicate specializers (level 3) are more specific than type specializers and type-predicate specializers (both level 2), which are more specific than destructuring patterns (level 1) and any-match (level 0). Type specializers `(x <type>)` and type-predicate specializers `((<type>? x))` are at the same level since both perform type checking. When multiple patterns at the same level could apply, first-match semantics determine the winner:

```
(defn check
  ([(x (= 5))] :exactly-five)      ; Predicate (
      level 3)
  ([(x <number>)] :some-number)    ; Type (level 2)
  ([x] :anything))                 ; Any (level 0)

(check 5)   ; => :exactly-five (pred > type)
(check 10)  ; => :some-number
(check "x") ; => :anything
```

Specificity ordering ensures that more constrained patterns are tested before less constrained ones, regardless of their definition order. This prevents common errors where a general pattern inadvertently shadows a specific one. When defining a function with multiple clauses, programmers need not worry about clause ordering for correctness—the system automatically tries the most specific matching pattern first. For sequence patterns, specificity extends to element-level comparisons: a pattern like `[(x (< 0)) y]` is more specific than `[(x <number>) y]` because its first element has higher specificity (predicate vs. type). This recursive specificity comparison enables precise control over dispatch behavior in complex nested destructuring scenarios.

5) Integration with fn and lambda: Predicate specializers work seamlessly in anonymous functions:

```
;; Inline predicate dispatch
((fn ([(x (< 10))] :small)
     ([(x (>= 10))] :large)) 5)
; => :small

;; Lambda with predicates
(def classifier
  (λ ([(n (< 0))] :neg)
     ([(n (> 0))] :pos)
     ([n] :zero)))
```

6) Restrictions and Design Rationale: Predicate specializers are **not allowed** in macro parameter lists because macros receive unevaluated forms. A predicate needs to evaluate its argument, but macros work with raw syntax:

```
;; This signals an error:
(defmacro bad-macro
  ([(x (= 0))] `0)
  ([x] `(inc ~x)))

; ERROR: Predicate specializers cannot be
;        used in DEFMACRO parameter lists
;        (macros receive unevaluated forms)
```

This restriction prevents confusion between compile-time pattern matching (on syntax) and runtime value testing (on evaluated data). The error is detected immediately during macro definition, providing clear feedback.

7) Implementation: Predicate specializers are compiled to a signature format `(:pred fn-name (arg0 arg1 ...))` during function definition. At runtime, pattern matching evaluates `(apply fn-name actual-arg arg0 arg1 ...)` and dispatches to the clause when the result is truthy. The implementation maintains O(N) dispatch time where N is the number of patterns at a given arity, with patterns sorted by specificity for efficient matching.

This design provides powerful dispatch capabilities while maintaining simplicity and integration with FOL's existing pattern matching infrastructure.

V. Self-Hosted Evaluator

FOL includes a meta-circular evaluator written in FOL itself (approximately 350 lines):

```
(defgeneric eval-form [form env])

(defmethod eval-form [(form <number>) env]
  form)  ; Self-evaluating

(defmethod eval-form [(form <symbol>) env]
  (lookup env form))

(defmethod eval-form [(form <list>) env]
```

```
  (if (empty? form)
    form
    (bind [op (first form)
           args (rest form)]
      (if (special-form? op)
        ((get special-forms (name op)) args env)
        (bind [fn-val (eval-form op env)
               evaled-args
                 (map #(eval-form % env) args)]
          (apply-function fn-val evaled-args))))))
```

The evaluator demonstrates several FOL features:

- Generic function dispatch on form types
- Pattern matching through method specialization
- First-class functions and lexical closures
- Special form handling through a dispatch table

Special form evaluators are defined as regular functions:

```
(defn eval-if [args env]
  (bind [arg-count (size args)]
    (if (or (< arg-count 2) (> arg-count 3))
      (throw "if: expected 2 or 3 args")
      (bind [test (nth args 0)
             then-form (nth args 1)
             else-form (if (>= arg-count 3)
                         (nth args 2)
                         nil)]
        (if (fol-eval test env)
          (fol-eval then-form env)
          (fol-eval else-form env))))))
```

The self-hosted evaluator serves multiple purposes:

1) **Specification**: Defines FOL semantics precisely
2) **Metaprogramming**: Enables runtime code generation
3) **Extensibility**: Allows user-defined special forms

Beyond these practical applications, the self-hosted evaluator validates FOL's design as a complete programming environment. A language that can implement its own evaluator demonstrates sufficient expressive power for complex metaprogramming tasks. The evaluator's relative brevity—350 lines implementing core evaluation semantics—speaks to FOL's abstraction capabilities.

The evaluator also serves pedagogical purposes. New FOL programmers can read the evaluator implementation to understand language semantics precisely. This executable specification complements informal documentation, resolving ambiguities through runnable code. Graduate courses on programming language implementation could use FOL's evaluator as a teaching vehicle, demonstrating how modern language features like pattern matching and persistent data structures enable elegant interpreter implementations.

From a practical standpoint, the evaluator enables powerful metaprogramming patterns. Domain-specific languages embedded in FOL can define custom evaluation semantics by extending or replacing the evaluator. Code generation tools can produce FOL forms and evaluate them dynamically. Testing frameworks can instrument the evaluator to track code coverage or inject debugging capabilities. These applications demonstrate that self-hosting provides not merely theoretical elegance but practical utility.

VI. Evaluation

A. Performance Characteristics

FOL's performance characteristics reflect its implementation strategy:

TABLE II
Complexity Comparison: Persistent vs. Mutable

Operation	Persistent	Mutable
Vector access	O(log n)	O(1)
Vector update	O(log n)	O(1)
Dict lookup	O(log n)	O(1)
Dict insert	O(log n)	O(1)
Set membership	O(log n)	O(1)

While persistent structures incur a logarithmic factor, the constant factors are small due to high branching factors (32-64). For typical applications, the difference is negligible compared to I/O and algorithmic complexity.

This performance profile makes FOL well-suited for applications where correctness, maintainability, and concurrent programming capabilities outweigh raw computational speed. Web services, data processing pipelines, and interactive development tools all benefit from persistent data structures' safety guarantees. The logarithmic overhead becomes insignificant compared to network latency, disk I/O, or algorithmic complexity of domain logic.

Importantly, FOL's interpreted execution currently dominates performance characteristics more than persistent data structure overhead. Profiling reveals that evaluation dispatch and environment lookups consume more cycles than collection operations. This suggests that compilation to native code or even to optimized Common Lisp would yield substantial speedups—potentially making FOL competitive with compiled Lisps while retaining persistence benefits. The FSet library demonstrates that persistent collections compiled to native code can achieve performance within 2-3x of mutable alternatives for many workloads.

B. Comparison with Related Languages

TABLE III
Feature Comparison

Feature	FOL	Clojure	CL
Persistent collections	✓	✓	×
CLOS/MOP	✓	×	✓
Transducers	✓	✓	×
Lazy sequences	✓	✓	×
Multi-pattern dispatch	✓	✓	×
Macros	✓	✓	✓
Reader syntax	Clojure	Clojure	CL

FOL occupies a unique position, providing both Clojure's functional features and Common Lisp's object system. This combination addresses limitations in both parent languages. Clojure programmers sometimes find protocols and records insufficient for complex object-oriented designs, particularly when inheritance hierarchies or method combinations would

clarify domain models. Common Lisp programmers, conversely, recognize the benefits of persistent data structures but lack native support, forcing them to choose between mutable performance and functional purity.

FOL demonstrates that these features complement rather than conflict. Persistent collections benefit from generic function dispatch—processing nested structures polymorphically becomes natural when methods can specialize on collection types. The MOP enables customization of persistence behavior, allowing classes to define specialized sharing strategies or optimization hooks. Transducers compose naturally with generic functions, enabling transformation pipelines that dispatch on element types.

This synthesis suggests that the historical divergence between functional and object-oriented Lisps may be reconcilable. Rather than viewing these paradigms as opposed philosophical camps, FOL treats them as complementary tools applicable to different aspects of program design. The success of this integration challenges assumptions about paradigm incompatibility that have influenced decades of language design.

VII. Future Work

Several extensions are planned for FOL:

- **Compilation**: Native code generation via transpilation to SBCL or native compilation
- **Class definition enhancements**: Abstract and sealed class definitions
- **Parallel collections**: Fork-join parallelism for sequence operations
- **Enhanced error system**: Condition classes, more powerful error-handling paradigms
- **Enhanced stream classes**: Additional stream classes, *in*/*out* streams
- **Storage options**: Refs with Software Transactional Memory, agents (atoms already implemented)
- **Core.async**: Communication and CPS capabilities

The most pressing need is compilation. Currently, all code is interpreted through the evaluator. Compiling to Common Lisp or native code would provide substantial performance improvements.

VIII. Related Work

A. Clojure

Clojure [**?**] pioneered persistent data structures in production Lisp. FOL adopts Clojure's sequence abstraction, transducer protocol and persistence while diverging in object system design. Where Clojure uses protocols and records, FOL uses CLOS generic functions and classes.

B. Common Lisp

FOL's object system is a direct descendant of CLOS [**?**]. The meta-object protocol [**?**] enables deep customization of class and generic function behavior. FOL extends this with persistent slot values.

C. Dylan

Dylan [**?**] influenced FOL's naming conventions and multiple dispatch semantics. The `<type>` notation improves code readability by clearly distinguishing types. In addition, FOL adopts Dylan's module system. Dylan modules provide a persistent way to package symbols. We believe it is also simpler and more consistent when compared with Clojure's namespaces and Common Lisp's packages.

D. Persistent Data Structures

Okasaki's work [**?**] on purely functional data structures underpins modern implementations. FSet [**?**] and Sycamore provide production-quality persistent collections for Common Lisp.

IX. Conclusion

FOL demonstrates that functional and object-oriented programming are synergistic. By combining Clojure's persistent data structures with CLOS's powerful object system, FOL enables new programming patterns unavailable in either parent language.

The language's key contributions include:

- Integration of persistent collections and objects with CLOS
- Self-hosted evaluator showcasing metaprogramming
- Unified syntax spanning multiple Lisp traditions
- Production-ready implementation in Common Lisp

FOL shows that the Lisp family's evolution need not be divergent. By thoughtfully combining ideas from different dialects, we can create languages that are more than the sum of their parts.

The complete FOL implementation, including 6,331 comprehensive tests achieving 100% pass rate and extensive documentation, is available at https://github.com/frankadrian/fol.

Acknowledgment

The author thanks the Common Lisp community for maintaining excellent libraries including FSet and Sycamore that made this implementation possible.

References

[1] G. L. Steele Jr., *Common LISP: The Language*, 2nd ed. Digital Press, 1990.

[2] D. G. Bobrow, L. G. DeMichiel, R. P. Gabriel, S. E. Keene, G. Kiczales, and D. A. Moon, "Common Lisp Object System Specification," *ACM SIGPLAN Notices*, vol. 23, no. SI, pp. 1–142, 1988.

[3] G. Kiczales, J. des Rivières, and D. G. Bobrow, *The Art of the Metaobject Protocol*. MIT Press, 1991.

[4] R. Hickey, "The Clojure programming language," in *Proceedings of the 2008 Symposium on Dynamic Languages*, 2008.

[5] Apple Computer, Inc., *Dylan Interim Reference Manual*. Apple Computer, Inc., 1993.

[6] R. Hickey, "Transducers," *Clojure Blog*, August 2014. Available: https://clojure.org/reference/transducers

[7] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1999.

[8] S. L. Siskind, "FSet: A functional set-theoretic collections library for Common Lisp," *Quicklisp*, 2012. Available: https://common-lisp.net/project/fset/